



Dino Run(디노 런) 기획서

이름: 이유승

작성일자: 26.05.15

플레이 영상: <https://youtu.be/aCCHo9AlxIs>

내용

- 게임 개요
- 게임 핵심 루프 및 조작 방식
- 게임 시스템 메커니즘
- 레벨 디자인
- UI/UX 연출
- 개발 데이터

● 게임 개요

게임 장르: 프리시전 플랫폼머(Precision Platformer)

게임 플랫폼: PC

▼ 프로젝트 개발 환경

- 게임 엔진: Unity 6 LTS
- 스크립트 에디터: Visual Studio Code (C# Extension 연동)
- 사용 그래픽스 API: DirectX 12 (DX12)
- 타겟 배포 플랫폼: PC Standalone (Windows 64-bit)
- 그래픽 / UI 리소스 디자인: 2D 픽셀 아트 소스 에셋 (Sprite 패키징)
- 활용 사이트: google Gemini

게임 목적(테마): 귀여운 공룡 디노를 플레이 하여 여러 장애물과 코인을 획득 하여 높은 점수를 기록한다.

시놉시스: 평화롭던 공룡 마을에 갑자기 거대한 위기가 찾아왔습니다! 다른 공룡들은 모두 대피했지만,
잠꾸러기 꼬마 공룡 '디노'는 뒤늦게 잠에서 깨어나고 말았습니다. 이제 디노는 밀려오는 위험을 피해
앞만 보고 달리기 시작합니다. 디노는 무사히 장애물들을 뛰어넘어 안전지대까지 도달할 수 있을까요?

●게임 핵심 루프 및 조작 방식

핵심 게임 루프 (Core Loop):장애물(선인장) 회피 및 점프 -> 코인 획득 및 점수 쌓기 -> 기록 갱신



게임 시스템 및 조작법

- **조작법:** A, D 또는 키보드 방향키 좌우 이동 / Spacebar 로 점프
- **입력 장치:** PC 키보드 전용 (키 매핑 표준화)



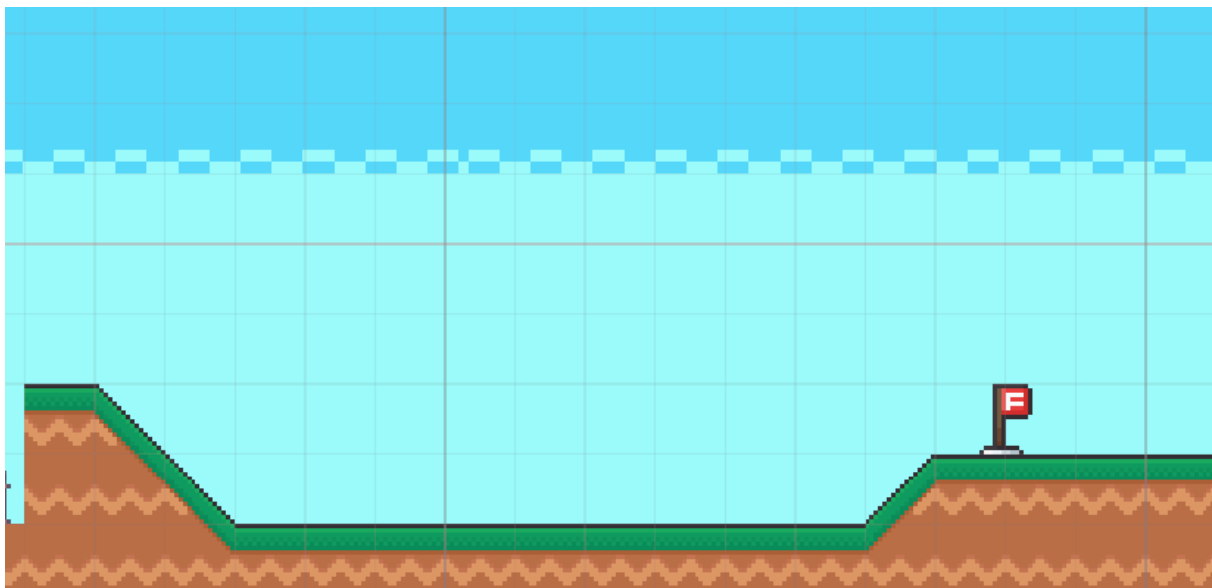
- **점프 메커니즘:** `Spacebar` 입력 시 즉시 상승 가속도 적용
단발성 단일 점프 지원 (공중 2단 점프 불가로 난이도 유지)
추락 및 점프 시 물리 엔진(`Rigidbody 2D` 중력 계수) 연동
스피드 값과 점프 값을 직접 입력하여 플레이어에게 더 나은 플레이 환경 제공

● 게임 시스템 메커니즘

- **거리 기반 점수 시스템**
플레이어가 제자리걸음을 하거나 뒤로 돌아갈 때 꼼수로 점수가 오르는 것을 방지하기 위해, '여태까지 이동한 거리 중 최고 X축 좌표'를 실시간으로 추적하여 점수로 환산 및 최고 기록 갱신
- 현재 캐릭터의 X 값 위치를 기억 한 뒤 (`currentDistance`)를 활용 하여 제자리에서 점수가 오르거나
뒤로 갈때 점수가 오르지 않고 오로지 앞으로만 전진할 때 점수가 오르도록 설정
- **게임 오버 조건**



달리는 도중 장애물에 충돌하거나, 발판 아래로 추락하면 게임이 종료되고 결과 창이 나타남
스테이지 클리어(승리) 조건

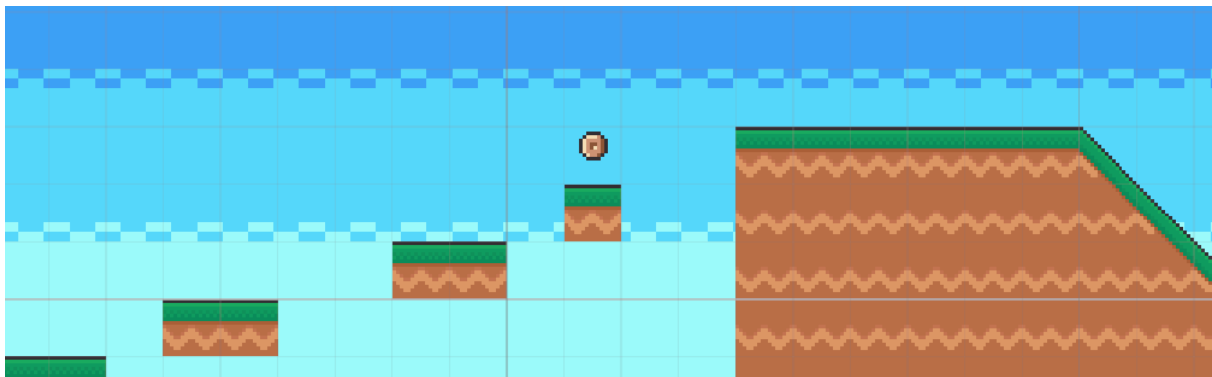


- **조건:** 플레이어가 장애물을 파훼하고 맵 최우측 끝단에 위치한 '**피니쉬 깃발(Finish)**' 오브젝트와 충돌 시 클리어 판정



- **클리어 연출:** 충돌 즉시 화면에 픽셀 아트 스타일의 **CLEAR!** 승리 팝업 UI 출력 및 플레이어 조작 일시 정지
- **포스트 루프:** 코루틴(Coroutine) 제어를 통해 승리 화면을 **3초** 간 유지하여 유저에게 성취감을 부여한 후, 해당 스테이지를 자동으로 처음부터 재시작

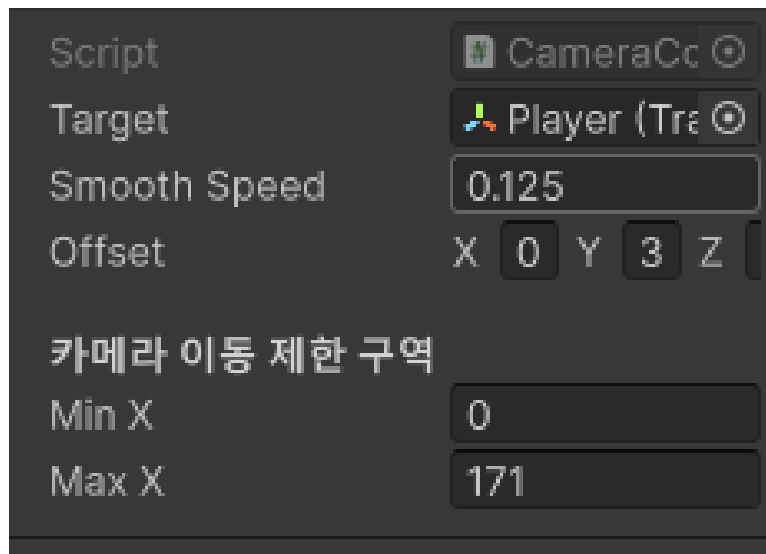
• 실시간 점수 배율 버프 (코인 시스템)



단순히 단순 합산으로 점수가 오르는 무료함을 탈피하기 위해 '점수 2배 뺑튀기' 코인 배치. 코인을 먹는 순간 현재까지 누적된 총점수가 즉시 2배로 폭등하여 플레이어에게 강력한 점수에 대한 성취감 제공

• 스마트 2D 카메라 시스템 (시야 제한):

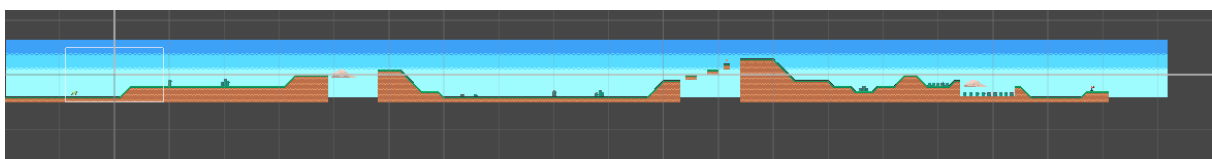
플레이어의 위치를 부드럽게 추적(`Vector3.Lerp`)하되, 맵의 경계선 밖 허공(배경 미지정 구역)이 화면에 노출되는 것을 방지



- `Mathf.Clamp` 함수를 이용해 카메라의 가로 이동 범위를 최소 시작점(Min X: 0)과 최종 끝인지점 (Max X: 171) 사이로 강제 제한
- 플레이어의 점프 구역에서 화면이 위아래로 심하게 흔들리는 멀미 현상을 방지하기 위해, 카메라의 높이와 거리를 정적으로 세팅하여 고정된 횡스크롤 뷰 제공

● 레벨 디자인

- 타일맵(Tilemap) 기반 지형 설계 (평지, 경사로, 공중 발판)

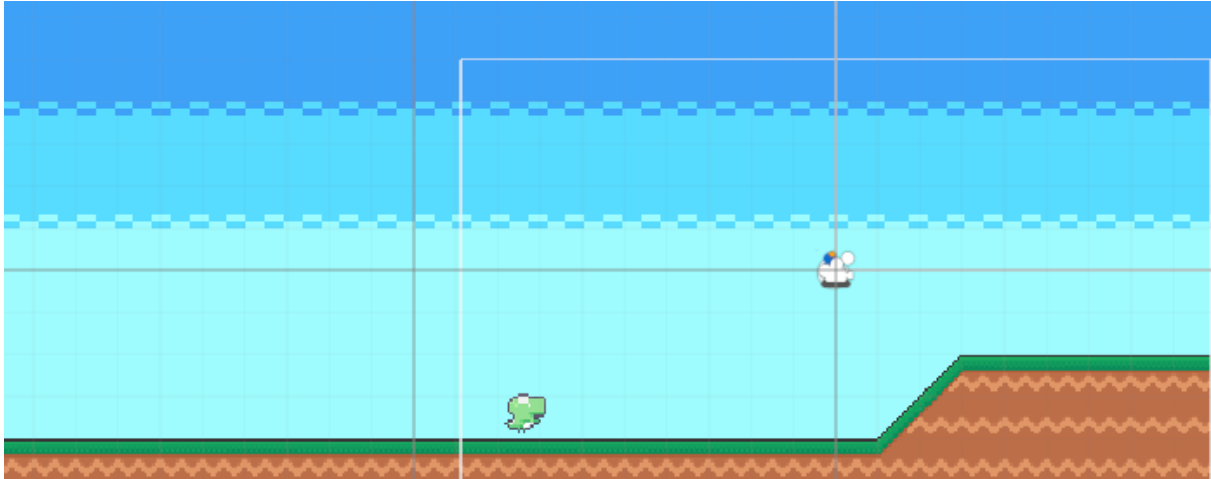


플레이어의 몰입감과 도전 욕구를 자극하기 위해 스테이지 전반의 난이도 곡선(완급 조절)을 고려하여

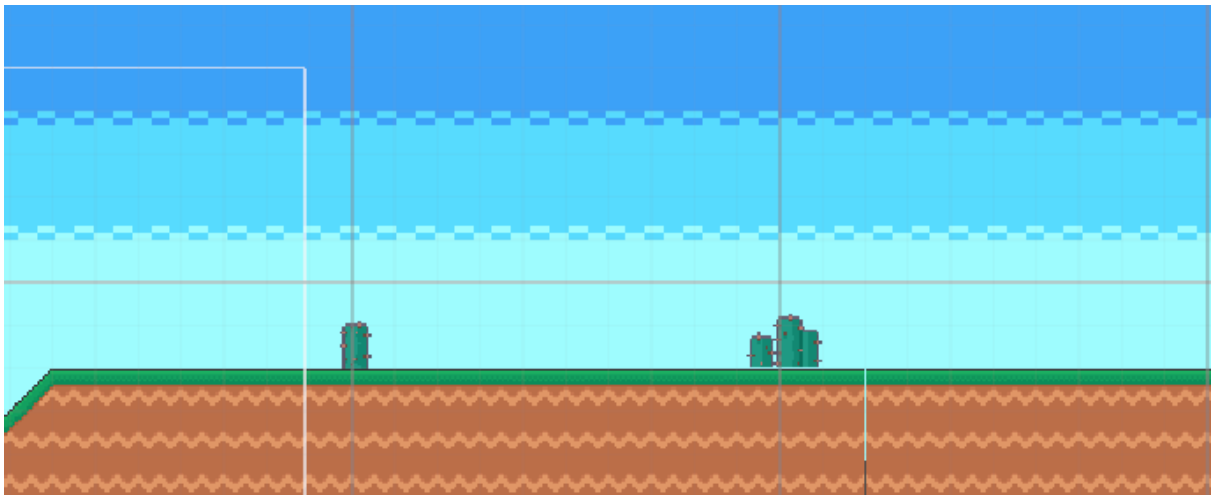
지형을 설계함

타일맵 기반 지형 설계를 통해 횡 스크롤 게임에 레벨 디자인 몰입감을 고조시킴

- 도입부(움직임 적응)

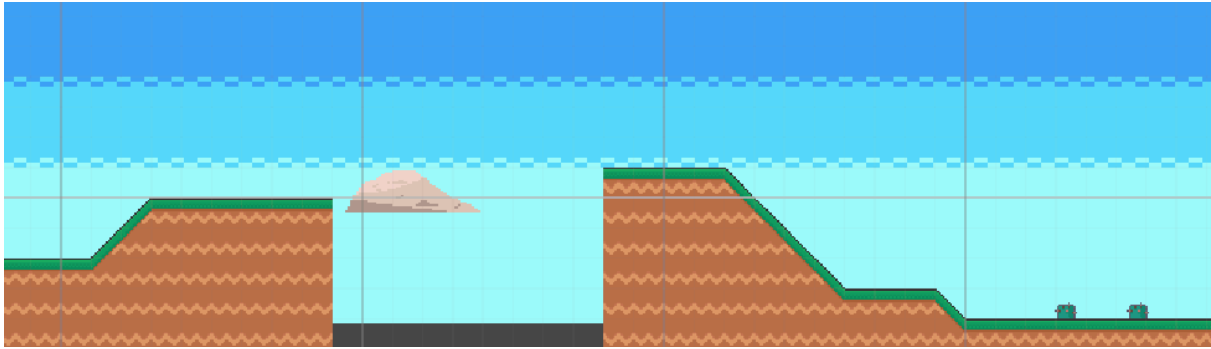


게임 시작 후 플레이어가 조작법(이동 및 점프 타이밍)에 익숙해질 수 있도록 장애물이 없는
평지 지형을
배치함



또한 간격이 넓은 장애물을 배치하여 장애물을 뛰어 넘을 때 점프 타이밍을 플레이어에게 익
힐 수 있도록 제작

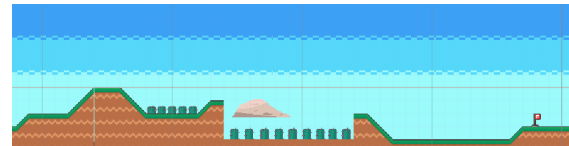
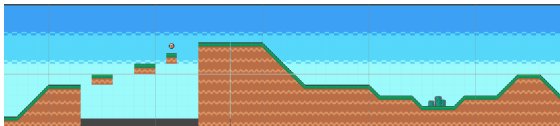
- 중반부 (경사로 및 입체 지형)



단순한 평지 반복으로 인한 지루함을 탈피하기 위해 랜드마크 형태의 **오르막길과 내리막길 경사로 지형**을

추가하고, 움직이는 발판을 추가 하여 플레이어에게 새로운 플레이 방식과 재미를 제공 경사로 지형으로 인한 점프 난이도 증가

- 후반부 (공중 점프 및 경사로, 선인장 다수 추가)

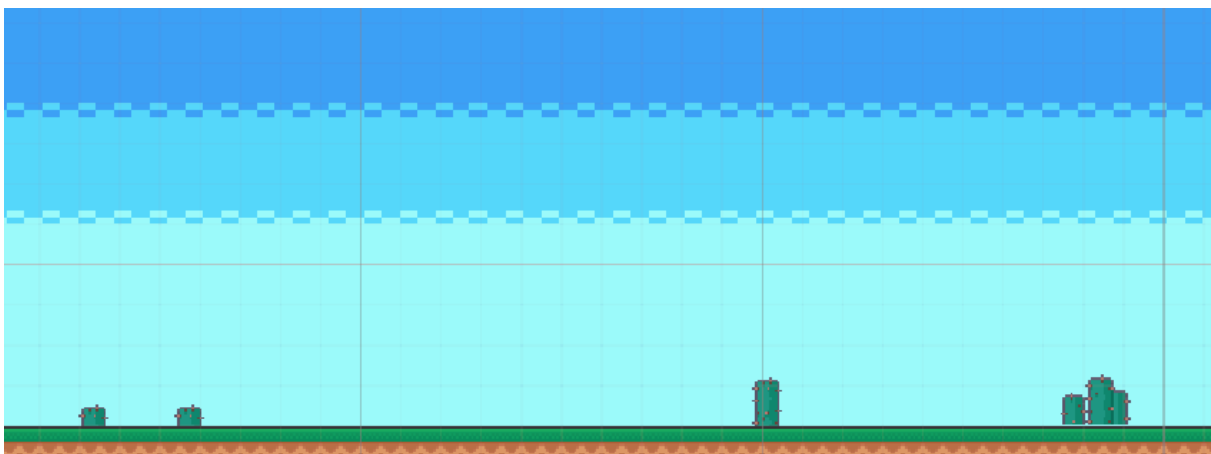


공중 발판을 제작 하여 점프를 통한 파훼로 진행 난이도 상승, 마지막 발판에 코인을 배치 하여 유저에게

도전 욕구 상승 높은 경사로 지형과 선인장 다수 추가로 더욱 정교한 점프 타이밍 요구

- 장애물 및 보상(코인) 배치 밸런싱

선인장(Cactus) 배치 패턴



선인장(Cactus) 배치 패턴

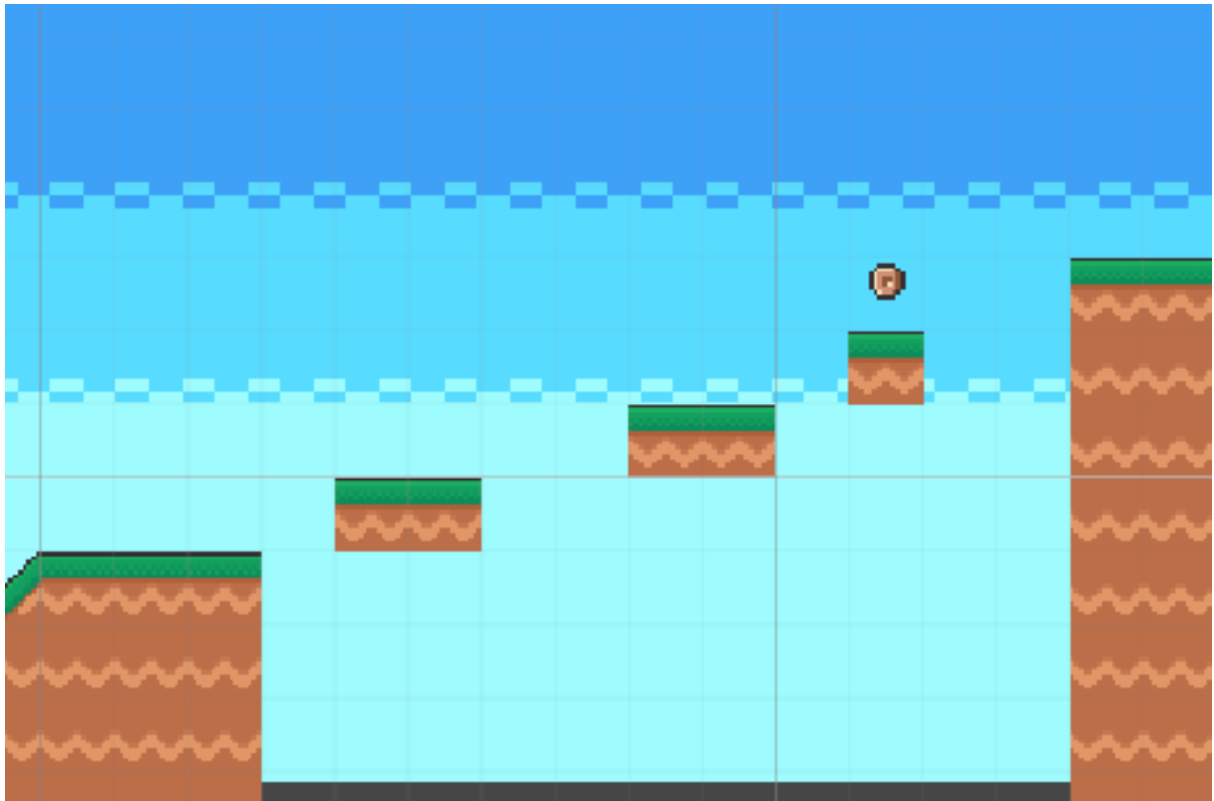
- 기본적으로 플레이어의 점프 최고 높이와 체공 시간을 계산하여 뛰어넘을 수 있는 간격으로 배치함
- 연속 점프를 유도하기 위해 선인장 여러 개를 연달아 배치하거나, 경사로 끝나는 지점 등 시야가 살짝 가려지는 곳에 배치하여 긴장감을 유도함.

하늘에서 떨어지는 돌 패턴



- 일정 위치에 도달하면 자동으로 플레이어를 쫓아오는 돌 생성 피격 시 사망
- 돌이 땅에 닿거나 플레이어 피격 시 자동 삭제, 플레이어 일정 거리 도달 시 돌 생성 멈춤

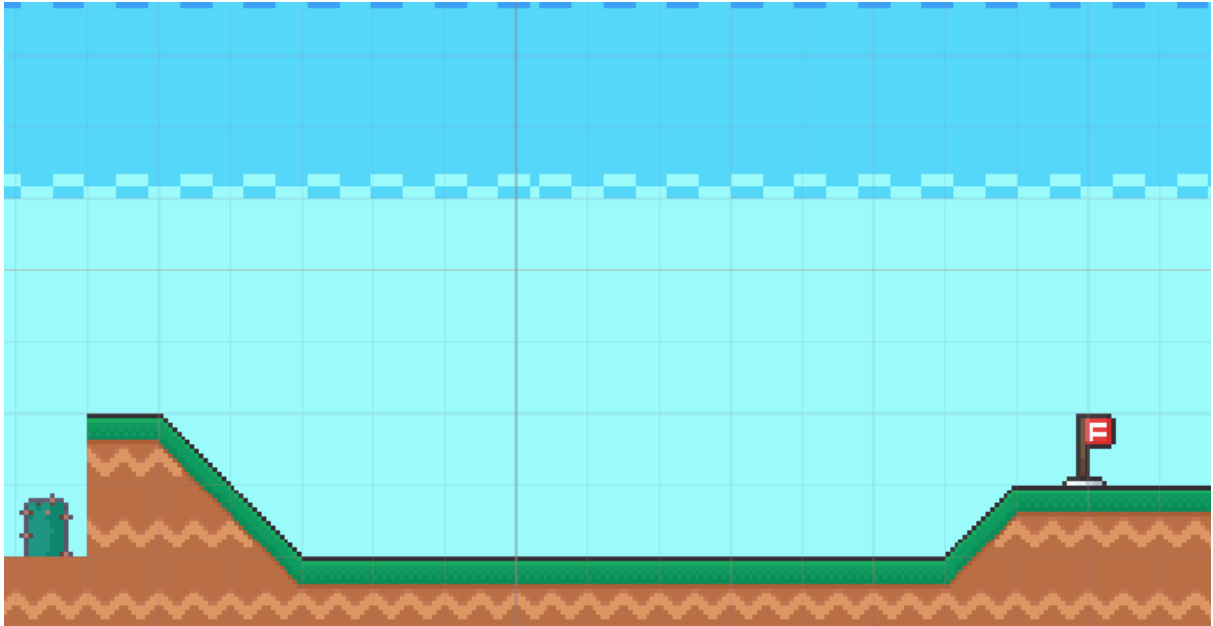
코인(Coin) 배치를 통한 보상 설계



코인(C)

- 낙사 하기 쉬운 공중 발판 점프 구간 마지막에 코인을 배치
- 기획 의도: 플레이어에게 공중 발판 점프를 클리어 했다는 성취감과 보상 심리를 동시에 주고 싶은 의도

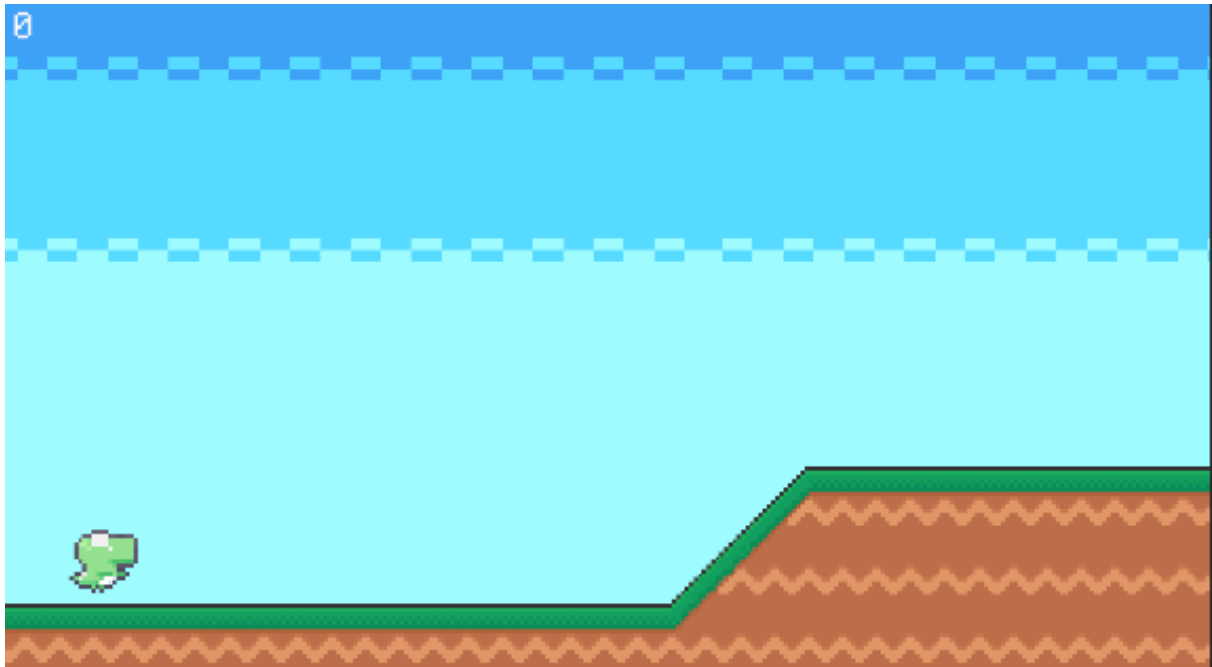
목표 지점 피니쉬(Finish)깃발 세팅



- **최종 목적지 구성:** 플레이어가 모든 장애물과 지형 돌파에 성공했을 때 명확한 시각적 피드백을 주기 위해 맵의 최우측 끝단(Max X 부근)에 '**피니쉬 깃발(Finish)**' 오브젝트를 배치함.
- 해당 깃발에 도달하는 순간 스테이지가 종결되며, 승리 연출(CLEAR! 팝업) 시스템으로 매끄럽게 전환되도록 트리거 범위를 설정함

UI/UX 연출

인게임 HUD (Heads-Up Display) 및 UI 배치 가이드



플레이어가 게임에 몰입하면서도 자신의 현재 상태를 직관적으로 인지할 수 있도록 화면 인터페이스를
정석적으로 배치

- 실시간 스코어 보드

위치: 인게임 화면 좌측 상단 (Score: 000 형태)

디자인: 가독성이 높고 게임의 레트로 도트 감성과 어울리는 픽셀 아트 스타일의 폰트 (TextMeshPro) 적용

UX 의도: 플레이어가 이동한 거리와 코인 버프 결과가 실시간으로 반영되는 것을 보며, 자신의 최고 기록을 갱신하려는 도전 욕구를 시각적으로 끊임없이 제공

스테이지 클리어 연출 및 UX 시나리오



스테이지를 완주했을 때 플레이어에게 확실한 성취감(Reward)을 제공하고, 자연스럽게 다음 플레이로

유도하는 연출 플로우를 구축

- 시각 연출 (Clear Popup UI)

피니쉬 깃발 충돌 즉시 화면에 픽셀 아트 디자인의 Clear 이미지가 활성화, 이때 플레이어는 조작이 일시정지 되어 게임이 끝났다고 인지함

- 시간 지연을 통한 감정적 UX 제어 (Coroutine)

클리어 이미지가 뜨자마자 게임이 푹 꺼지거나 바로 재시작되면 유저가 허탈함을 느낄 수 있으므로, 유니티 **Coroutine** 을 활용해 정확히 3.0초간 클리어 화면을 유지

이 3초의 대기 시간은 플레이어가 스테이지를 완수했다는 시각적 성취감을 온전히 느끼게 만드는

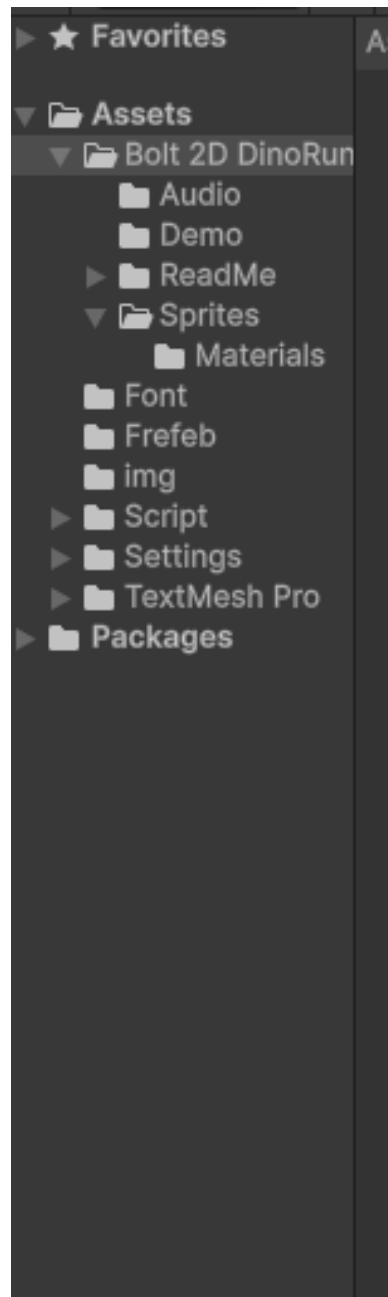
핵심 여백 역할을 함

- 씬 재시작 루프 (Scene Reload):

3초가 지나면 **GameManager** 를 통해 현재 스테이지를 자동으로 리로드하여 유저가 별도의 조작(클릭 등)을 하지 않아도 곧바로 다음 도전을 이어갈 수 있는 '멈춤 없는 무한 플레이 루프' 환경을 제공

개발 데이터

- 프로젝트 디렉터리 및 에셋 구조

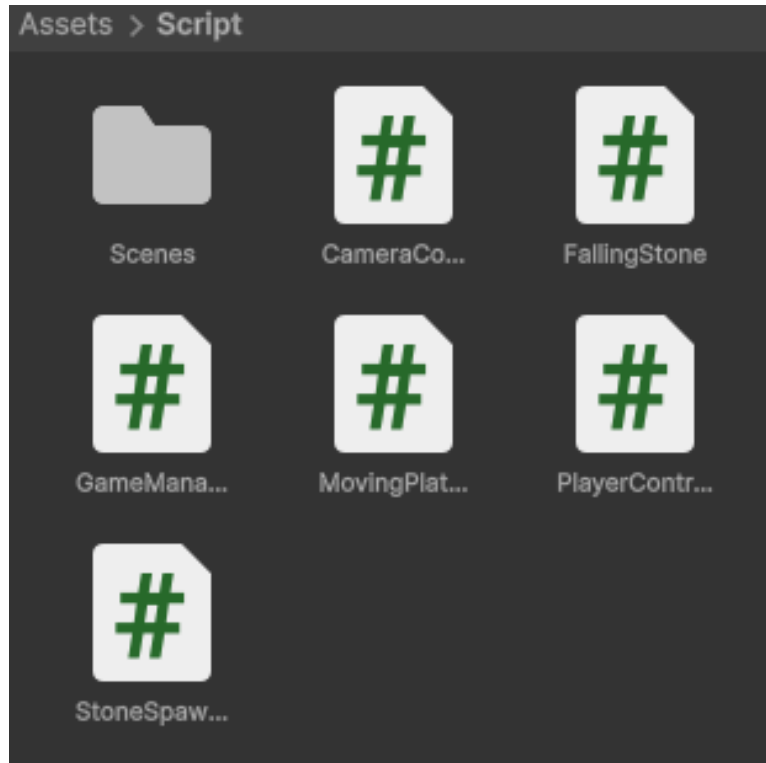


• Assets 폴더 내 폴더 구조화

- **/_Bolt 2D DinoRun** : 본 프로젝트의 메인 개발 루트 폴더
- **/Audio** : 배경음악(BGM) 및 코인 획득, 점프, 게임오버 관련 사운드 에셋 보관
- **/Font** : 인게임 HUD 점수판 표시에 사용되는 픽셀 아트 스타일의 TextMeshPro 전용 폰트 에셋 관리
- **/Img & /Sprites** : 디노 캐릭터 애니메이션, 선인장 장애물, 타일맵 텍스처 및 UI 소스 관리
- **/Scenes** : 메인 플레이 스테이지가 저장된 **SampleScene.unity** 보관

- **/Script** : 객체 지향 설계를 기반으로 분할된 C# 개발 스크립트 파일 일체 저장

• 주요 스크립트 역할 정의



• PlayerController.cs (플레이어 제어 및 충돌)

- 키보드 입력을 처리하여 캐릭터의 수평 이동 및 점프 물리 동작 제어
- **OnTriggerEnter2D** 트리거 충돌 감지 시스템을 통해 코인 획득 처리 및 피니쉬 깃발 충돌 시 Clear 시퀀스 발동 역할 전담

• GameManager.cs (게임 상태 및 데이터 관리)

- 실시간 스코어 연산 및 UI 텍스트(TextMeshProUGUI) 바인딩 세팅
- 코루틴(**IEnumerator**) 시스템을 내장하여 깃발 충돌 시 3초의 비동기 시간 지연 후 **SceneManager** 를 통한 스테이지 재시작 흐름 총괄

• CameraController.cs (카메라 뷰 및 시야 제한)

- 디노의 X축 좌표 추적 및 연산
- **Mathf.Clamp** 알고리즘을 사용해 시야의 최소/최대 경계를 고정하여 비정상적인 외부 공백 노출을 원천 차단

● FallingStone.cs (떨어지는 돌 오브젝트 제어)

- **OnCollisionEnter2D** 기반의 물리적 충돌 처리
- 돌 오브젝트가 바닥 지형(`wall` 등)에 충돌하는 순간을 `OnCollisionEnter2D` 로 감지
- 충돌 발생 시 플레이어의 생명력을 즉시 0으로 판단하고, `GameManager.cs` 를 호출하여 인 게임 시스템 상의 '게임 오버 시퀀스(결과 창 팝업 및 플레이 중단)'를 즉각 발동

● StoneSpawner.cs (동적 장애물 생성 및 추격)

- 플레이어가 스폰너를 지나치기 전에는 작동하지 않다가, 플레이어의 X축 좌표가 스폰너의 X축 좌표를 추월하는 순간(`distanceAfterPassing >= 0`)을 실시간 연산함
- 스폰너가 활성화되면 프레임 독립 속도(`Time.deltaTime`)와 지정 속도(`moveSpeed = 3f`)를 연산하여 지정된 최종 종착지(`endPositionX = 120f`)까지 수평 이동하며 플레이어를 추격함

● MoveinPlatform.cs (움직이는 발판 제어)

- 매 프레임마다 발판의 이전 위치(`previousPosition`)와 이동 후 현재 위치의 차이 값을 실시간으로 계산
- 프레임 독립적인 속도 제어(`Time.deltaTime`)를 활용해 상하/좌우 왕복 운동의 프레임 간 실제 이동 벡터(거리)를 정밀하게 추출함
- 플레이어가 움직이는 발판에 착지하여 콜라이더가 접촉하는 최초 순간(`OnCollisionEnter2D`)을 감지, 디노의 최상위 오브젝트를 탑승객으로 즉시 등록

● 인 게임 에셋 및 오브젝트 명세표

오브젝트명	태그(Tag)	컴포넌트 구성	주요 기획 수치 및 역할
Player (디노)	<code>Player</code>	Rigidbody 2D, Collider 2D, PlayerController.cs	이동 속도(5f), 점프력(10f), 주인공 캐릭터
Cactus (선인장)	<code>Obstacle</code>	Box Collider 2D (Is Trigger)	충돌 시 즉시 게임 오버 및 결과창 팝업 유도
Coin (코인)	<code>Coin</code>	Box Collider 2D (Is Trigger)	충돌 시 현재 점수 2배 폭등 버프 제공 후 파괴
Finish (깃발)	<code>Finish</code>	Box Collider 2D (Is Trigger)	스테이지 최종 목적지. 충돌 시 3초 코루틴 승리 연출 실행
Stone (떨어지는 돌)	<code>Obstacle</code>	Rigidbody 2D, Circle Collider 2D	하늘에서 떨어지는 동적 장애물. 플레이어와 충돌 시 즉시 게임 오버 유

			도, 바닥 지형 충돌 시 자동 파괴 처리.
Stone Spawner (스톤 스폰너)	Untagged	StoneSpawner.cs	플레이어가 자신을 추월했을 때 (distanceAfterPassing >= 0) 작동 시작하여, 이동 속도 3f로 쫓아가며 0.5초 간격으로 돌을 투하하는 동적 함정 장치.
Moving Platform (움직이는 발판)	Untagged	Box Collider 2D, MovingPlatform.cs	속도 3f, 이동 거리 5f 사양의 왕복 발판. 플레이어가 착지 시 탑승객 (passengerTransform)으로 등록하여 미끄러짐 방지 및 이동 좌표 실시간 동기화.